VybeSchool — Python & Backend Engineering for AI

30-Day Live Curriculum | Mentor 1

Duration: 30 live sessions
Class: 1–1.5 hours/session
Level: Beginner → Intermediate Engineering
Projects: 2 mini-projects + 1 major backend project
Outcome: Ready to begin the AI Engineering track immediately after Day 30.

PHASE 1 — PYTHON FOUNDATIONS

Days 1–7

Day 1 — Programming, Python & Developer Setup

Goal: A complete beginner should understand what they're actually doing when they write and execute code.

Concepts

- What programming actually means
- How source code becomes an executing program
- Why Python is heavily used in AI engineering
- Python interpreter
- Installing Python
- VS Code setup
- Terminal/command-line basics
- Creating `.py` files
- Running Python from terminal
- `print()`
- Comments
- Variables
- Naming conventions
- Basic debugging mindset
- Understanding errors instead of fearing the

Live Build

Create:

`student_profile.py`

Students take basic information and generate a formatted developer profile.

Practice

- Name
- Age
- Current role
- AI career goal
- Years until target career date

Assignment

Build a simple **AI Career Profile Generator**.

Day 2 — Data Types, Input & Operators

Concepts

- Integers
- Floats
- Strings
- Booleans
- `None`
- `type()`
- Type conversion
- User input
- Arithmetic operators
- Comparison operators
- Logical operators
- Assignment operators
- String formatting
- f-strings
- Common type errors

Live Build

AI Course Fee Calculator

Inputs:

Course price → discount → GST/fees if applicable → installments → final amount.

Engineering habit

Never trust input blindly.

Day 3 — Conditions & Decision Making

Students now make programs behave differently based on information.

Concepts

- `if`
- `elif`
- `else`
- Nested conditions
- Boolean expressions
- `and / or / not`
- Truthy/falsy values
- Membership operators
- Guard conditions
- Basic input validation

Live Build

Candidate Eligibility Checker

Input:

Python skill
Experience
Availability
Interest

Output:

Eligible / Needs Preparation / Not Eligible

This starts connecting coding to actual business logic.

Day 4 — Loops & Repetitive Processing

Concepts

- Why loops exist
- `for`
- `while`
- `range()`
- Iterating over strings/lists
- Nested loops
- `break`
- `continue`
- Loop conditions
- Counters
- Accumulators
- Avoiding infinite loops

Live Build

Process a list of student scores and generate:

Pass count
Fail count
Average
Highest score
Lowest score

Challenge

Build a basic bulk lead-processing script.

Day 5 — Python Collections

This is a **very important day for AI engineering** because JSON/API responses depend heavily on these structures.

Concepts

- Lists
- Tuples
- Sets
- Dictionaries
- When to use each
- Indexing
- Slicing
- Updating collections
- Nested dictionaries
- Lists of dictionaries
- Iterating collections
- Membership
- Common collection methods

Introduce structures resembling API responses:

```
student
 ├── id
 ├── name
 ├── skills[]
 └── projects[]
```

Live Build

Student Management System — V1

Create/update/search student records in memory.

Day 6 — Functions & Reusable Code

This needs strong attention.

Concepts

- Why functions matter
- Defining functions
- Parameters
- Arguments
- Positional arguments
- Keyword arguments
- Default arguments
- Return values
- Multiple return values
- Scope
- Local/global variables
- Pure functions
- Reusable logic
- Basic type hints

Take previous messy scripts and refactor them into functions.

Live Build

Turn Student Management V1 into modular functions:

```
add_student()
find_student()
calculate_score()
update_student()
generate_report()
```

Day 7 — Modules, Packages & Python Project Structure

Now move from scripts toward applications.

Concepts

- What modules are
- `import`
- Creating custom modules
- Python packages
- `__init__.py`
- Standard library
- Third-party packages
- `pip`
- `venv`
- `requirements.txt`
- Separating concerns
- Basic folder structure

Example:

```
student_app/
    main.py
    services/
    models/
    utils/
    data/
```

Students restructure their existing project.

Week 1 checkpoint

Student should now comfortably read/write basic Python without copying every line from the mentor.

PHASE 2 — PYTHON FOR REAL APPLICATIONS

Days 8–13

Day 8 — Files, JSON & CSV

Extremely important before AI/RAG.

Concepts

- File paths
- Reading files
- Writing files
- `with open()`
- TXT
- JSON
- CSV
- Serialization/deserialization
- Encoding basics
- Handling missing files
- Working with structured data

Live Build

Read:

`students.csv`

Process records.

Generate:

`report.json`

AI connection

Explain that later:

PDF/document → processing → chunks → embeddings → database

starts with understanding data and files.

Day 9 — Errors, Exceptions & Defensive Programming

Students must learn to expect failure.

Concepts

- Syntax errors
- Runtime errors
- Logical errors
- Exceptions
- `try`
- `except`
- `else`
- `finally`
- Raising exceptions
- Custom validation
- Defensive programming
- Debugging traceback
- Useful error messages

Live exercise

Break yesterday's application intentionally and make it resilient.

Examples:

Missing file
Wrong type
Invalid JSON
Empty values
Bad input

Day 10 — OOP for AI/Backend Developers

Do **not** make this a university OOP lecture.

Teach practical OOP.

Concepts

- Class
- Object
- Attributes
- Methods
- Constructor
- `self`
- Encapsulation
- Inheritance — basic
- Composition
- When classes help
- When functions are enough

Examples:

```
User
Document
Course
APIClient
```

Later these concepts translate naturally to:

```
LLMClient
Retriever
Agent
Tool
```

Live Build

Refactor part of existing application using classes.

Day 11 — Clean Python, Type Hints & Engineering Practices

This differentiates **“I know Python”** from **“I can work on a codebase.”**

Concepts

- PEP 8 awareness
- Naming
- Functions doing one thing
- DRY
- Separation of concerns
- Type hints
- Docstrings
- Constants
- Configuration
- Avoiding hard-coded values
- Basic code organization
- Refactoring

Bad code → clean code exercise.

Students review each other's small code samples.

Day 12 — HTTP, APIs & JSON

One of the **most important days in the entire first 30 days**.

Before FastAPI, students must understand the web.

Concepts

- Client/server architecture
- What happens when you open a website
- HTTP
- Request
- Response
- URL
- Endpoint
- HTTP methods:
    - GET
    - POST
    - PUT/PATCH
    - DELETE
- Headers
- Request body
- Query parameters
- Path parameters
- Status codes
- JSON
- API
- REST
- Authentication concept
- API keys

Use Postman/Bruno/cURL conceptually.

Show an actual public API response.

Day 13 — Consuming External APIs with Python

Now Python talks to other systems.

Concepts

- `requests`
- GET requests
- POST requests
- Headers
- Parameters
- JSON responses
- API keys
- Environment variables
- `.env`
- Timeouts
- Error handling
- Status-code handling
- Parsing responses
- Separating API client logic

Live Build

Python application → external API → process response → store result.

This is the direct precursor to:

Python → OpenAI/Anthropic API

in the AI track.

PROJECT 1

Python Automation System

Students now build their first proper mini-project.

Day 14 — Project 1 Build Day

Project: Automated Data Processing & Reporting System

Pipeline:

CSV / JSON
↓
Read
↓
Validate
↓
Process
↓
External API
↓
Generate Results
↓
JSON / CSV Report
↓
Logs

Must include:

- Functions
- Files
- JSON/CSV
- Error handling
- External API
- Environment variables
- Logging basics
- Clean project structure
- Git repository
- README

Mentor spends the session **building/debugging with students**, not introducing another chapter.

PHASE 3 — GIT & SOFTWARE WORKFLOW

Days 15–16

Day 15 — Git & GitHub

Concepts

- Why version control exists
- Git vs GitHub
- Repository
- `git init`
- `git status`
- `git add`
- `git commit`
- Commit messages
- GitHub repositories
- Push/pull
- Clone
- `.gitignore`
- README
- Protecting `.env`
- Never committing API keys

Every student publishes Project 1 properly.

Day 16 — Branching, Collaboration & Professional Git Workflow

Concepts

- Branches
- Feature branches
- Merge
- Merge conflicts
- Pull requests
- Code reviews
- Issues
- Basic collaboration
- GitHub project presentation

Simulation:

main
├── feature/auth
├── feature/database
└── feature/api

Students make a branch → change → PR → review → merge.

From today onward, **every assignment goes through GitHub.**

PHASE 4 — DATABASES FOR AI ENGINEERS

Day 17 — Database Fundamentals + SQL

Start with the problem databases solve.

Concepts

- Persistent data
- Relational databases
- PostgreSQL
- Database
- Table
- Row
- Column
- Data types
- Primary key
- Foreign key
- Constraints
- Basic normalization
- Schema design

SQL:

```
CREATE
INSERT
SELECT
UPDATE
DELETE
```

Live Build

Create a simple student/course database.

Day 18 — Practical SQL

Concepts

- `WHERE`
- `ORDER BY`
- `LIMIT`
- `LIKE`
- Aggregations
- `COUNT`
- `SUM`
- `AVG`
- `GROUP BY`
- Joins
- Inner/left joins
- Index concept
- Query performance concept

Use realistic data.

No unnecessary database theory.

Day 19 — Python + PostgreSQL + ORM

Move from writing SQL manually to application integration.

Concepts

- Python/database connection
- ORM concept
- SQLAlchemy
- Models
- Sessions
- CRUD
- Mapping classes → tables
- Relationships
- Connection configuration
- Environment variables

Students connect Python application to PostgreSQL.

Day 20 — Data Modeling for Real Applications

Very useful preparation for AI apps.

Design:

User
↓
Conversation
↓
Message

and:

User
↓
Document
↓
Processing Status

Concepts

- One-to-one
- One-to-many
- Many-to-many concept
- Foreign keys
- Constraints
- Timestamps
- IDs
- Schema decisions
- Basic migrations
- Alembic introduction

Students design a schema before coding it.

Day 21 — Caching, Redis & Application Data Flow

Keep Redis intentionally light.

Concepts

- Why databases aren't used for everything
- Persistent vs temporary data
- Cache
- Redis concept
- Key-value storage
- TTL
- Sessions
- Rate-limit counters
- Where Redis appears in AI systems

Then show:

Client
↓
Backend
↓
Cache
↓
Database

No advanced Redis internals.

PHASE 5 — BACKEND ENGINEERING WITH FASTAPI

Days 22–27

Day 22 — Building Your First FastAPI Backend

Concepts

- Backend recap
- FastAPI setup
- Application instance
- Routes
- GET
- POST
- Path parameters
- Query parameters
- Request bodies
- Response bodies
- Swagger/OpenAPI
- Running with Uvicorn

Students build:

```
/users
/courses
/health
```

Day 23 — Pydantic, Validation & API Design

Concepts

- Pydantic models
- Request schemas
- Response schemas
- Validation
- Optional fields
- Defaults
- Nested schemas
- Status codes
- HTTP exceptions
- API error responses
- REST naming conventions
- API versioning concept

Transform:

“API that works” → “API that is properly designed.”

Day 24 — FastAPI + PostgreSQL

Connect everything students have learned.

Architecture:

Client
↓
FastAPI
↓
Service Logic
↓
SQLAlchemy
↓
PostgreSQL

Concepts

- Database dependency
- Models
- Schemas
- CRUD
- Service layer concept
- Repository concept — light
- Relationships
- Error handling

Live Build

Create users → retrieve users → update users → delete users.

Day 25 — Authentication & Security Fundamentals

Concepts

- Authentication vs authorization
- Password hashing
- Never storing plain passwords
- Registration
- Login
- JWT
- Access tokens
- Protected endpoints
- API keys
- Secrets
- Environment variables
- Basic CORS awareness
- Basic security habits

Flow:

Register
↓
Hash Password
↓
Database

Login
↓
Verify
↓
JWT
↓
Protected API

Students implement basic auth.

Day 26 — Async Python + External Services

Now introduce async because students finally have a reason to care.

Concepts

- Synchronous execution
- Blocking operations
- Asynchronous execution
- `async`
- `await`
- Event-loop concept
- Concurrent external calls
- Async endpoints
- Timeouts
- Retries concept

Show:

```
Request
 ├── Database
 ├── External API
 └── Another Service
```

This becomes extremely important when Mentor 2 introduces multiple LLM/tool calls.

Day 27 — Testing, Logging & Production-Style APIs

Testing

- Why tests exist
- Unit vs integration concept
- `pytest` basics
- Testing functions
- Testing API endpoints
- Happy path
- Failure cases

Logging

- Why `print()` isn't enough
- Log levels
- INFO
- WARNING
- ERROR
- Debugging production problems
- Structured logging concept

Reliability

- Timeouts
- Retries
- Validation
- Exception handling

Project 2 checkpoint

Students should now have a functional:

Authenticated FastAPI + PostgreSQL application.

PHASE 6 — SHIPPING SOFTWARE

Days 28–30

Day 28 — Docker & Deployment Fundamentals

Students need to understand how software leaves their laptop.

Concepts

- Local vs production
- Server
- Cloud basics
- Container concept
- Why Docker exists
- Image vs container
- Dockerfile
- Dependencies
- Ports
- Environment variables
- `.dockerignore`
- Build
- Run

Students Dockerize their FastAPI application.

No Kubernetes.

No complex DevOps.

PROJECT 3 — MAJOR BACKEND PROJECT

Day 29 — AI-Ready Backend Build Day

This should be almost entirely practical.

Major Project: AI-Ready SaaS Backend

Architecture:

CLIENT → FASTAPI → AUTH + SERVICES + EXTERNAL API → POSTGRESQL → REDIS

Required Features

User system

- Registration
- Login
- Password hashing
- JWT

Backend

- REST endpoints
- CRUD
- Validation
- Pydantic schemas
- Service logic
- Error handling

Database

- PostgreSQL
- SQLAlchemy
- Relationships
- Basic migrations

External integration

- At least one external API

Engineering

- Environment variables
- Logging
- Basic tests
- Git/GitHub
- README

Mentor spends the class doing:

architecture review → live debugging → code review → Q&A.

Day 30 — Deploy, Review & Bridge to AI Engineering

This day should feel important.

Part 1 — Deployment

Deploy the application.

Students learn:

- Production environment variables
- Database configuration
- Docker deployment
- Logs
- Basic debugging
- Health endpoint
- Testing live endpoints
- Swagger in deployed environment

Part 2 — Engineering Review

Every student should be able to explain:

How does my request reach my backend?

How does authentication work?

Where is the data stored?

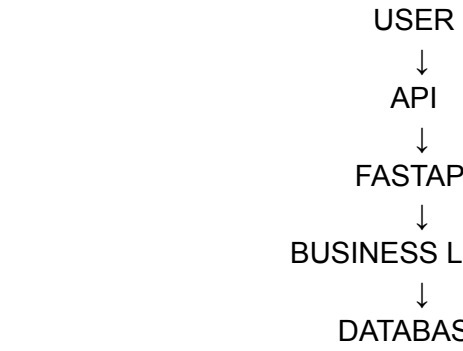
How does Python communicate with another API?

What happens when something fails?

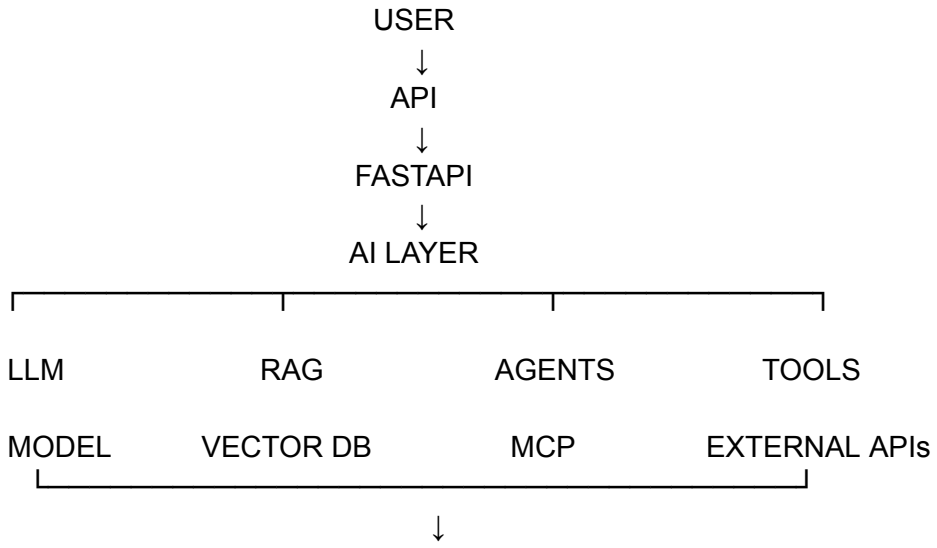
How would another application consume my API?

Part 3 — AI Transition

Mentor shows the architecture they've created:



Then reveal what's coming next:



For 30 days, you learned how to build software.

From tomorrow, you'll learn how to make that software intelligent.

VybeSchool — AI Engineering Track

Day 31 → Day 60 | 30 Live Sessions

Prerequisite: Students have completed the first 30 days covering Python, Git/GitHub, SQL/PostgreSQL, FastAPI, REST APIs, authentication, Docker and backend deployment.

Class duration: 60–90 minutes
Teaching style: approximately **30 min concepts + 25–35 min live building + 10–15 min debugging/review**.

The principle throughout this track is:

Understand → Build from Scratch → Use Framework → Evaluate → Productionize.

PHASE 1 — LLM ENGINEERING FOUNDATIONS

Days 31–35

DAY 31 — How LLMs Actually Work + First LLM Application

Goal

Students should understand what they're actually interacting with before using frameworks.

Concepts

- AI vs ML vs Deep Learning vs Generative AI
- Foundation models
- Large Language Models
- Parameters
- Training vs inference
- Pretraining
- Instruction tuning
- Alignment concept
- Transformer architecture at practical level
- Tokens
- Tokenization
- Context windows
- Input/output tokens
- Temperature
- Top-p
- Max output tokens
- Deterministic vs creative generation
- Hallucination
- Why LLMs predict rather than “search”
- Closed/API models vs open-weight models
- GPT / Claude / Gemini / Llama / Mistral model families conceptually
- Model selection based on:
    - quality
    - latency
    - context
    - price
    - privacy
    - modality

Hands-on

First direct model call using Python.

Students configure:

- API key
- environment variables
- SDK

- model
- system instruction
- user input
- response

Then experiment with:

temperature → instructions → context → output length.

Live Build

CLI AI Assistant

User → Python → Model API → Response

Engineering lesson

Show students the actual request and response objects instead of hiding everything behind LangChain.

Assignment

Create an AI assistant with:

- System instructions
- Multiple user inputs
- Conversation history
- Error handling
- Environment variables
- Token/usage logging

DAY 32 — Prompt & Context Engineering

Prompting should be taught as **engineering**, not “50 secret ChatGPT prompts.”

Concepts

- System instructions
- User messages
- Assistant messages
- Instruction hierarchy
- Zero-shot
- Few-shot
- Examples
- Delimiters
- Context
- Constraints
- Role prompting
- Output contracts
- Prompt templates
- Prompt variables
- Prompt injection concept
- Context engineering
- Why context often matters more than clever wording
- Reasoning-model prompting considerations
- Prompt versioning
- Prompt testing

Hands-on

Take one task and intentionally create:

1. Bad prompt
2. Improved prompt
3. Few-shot prompt
4. Context-rich prompt
5. Structured instruction

Compare outputs.

Live Build

Resume → Job Description Analyzer

Input:

Resume + JD

Output:

- matching skills
- missing skills
- explanation
- improvement suggestions

Job skill

Students should be able to debug:

> “Is my model bad, or is my context/instruction bad?”

DAY 33 — Structured Outputs, Function Calling & Tool Use

This is one of the most important LLM application-development sessions.

Concepts

- Why free-form text breaks applications
- JSON responses
- Schemas
- Structured outputs
- Pydantic integration
- Type-safe AI outputs
- Function/tool definitions
- Tool schemas
- Tool arguments
- Model choosing a tool
- Application executing the tool
- Returning tool result
- Model producing final response
- Multiple tools
- Validation/failure handling

Architecture

User → LLM → Tool Decision → Python Function/API → Tool Result → LLM → Structured Response

Hands-on

Create tools such as:

- `get_weather()`
- `search_database()`
- `get_customer()`
- `create_ticket()`

The important lesson is that the LLM **doesn't magically execute your Python function**.

Students manually implement the execution loop.

Live Build

AI Customer Support Assistant with Tools

Tools:

Customer lookup
Order lookup
Ticket creation

Assignment

Build a 3-tool assistant without LangChain.

DAY 34 — Open-Source Models, Hugging Face & Local AI

Now students leave API-only AI.

Concepts

- Proprietary models vs open-weight models
- Hugging Face ecosystem
- Model Hub
- Model cards
- Parameters
- Model architecture
- Tokenizers
- Checkpoints
- Safetensors concept
- Base vs instruct models
- Model licenses
- Model size
- VRAM/RAM
- CPU vs GPU inference
- Precision:
    - FP32
    - FP16/BF16
    - INT8
    - INT4
- Quantization
- GGUF concept

Hands-on

Use Hugging Face:

- Find a model
- Read model card
- Download/load model
- Load tokenizer
- Tokenize input
- Run inference
- Decode output

Then compare API model vs local/open model.

Live Build

Local Text Generation Application

No OpenAI API required.

Student outcome

They should understand what:

> “Download a model and run inference”

actually means.

DAY 35 — Ollama, Local Serving, Quantization & Model Routing

Concepts

- Ollama
- Local model runtime
- Pulling models
- Running local models
- Model configuration
- Quantized models
- Local API endpoints
- Streaming
- Model serving
- vLLM introduction
- Throughput
- Batching concept
- GPU serving concept
- API model vs self-hosted model
- Privacy
- Cost
- Latency
- Model routing
- Fallback models

Hands-on

Run local model with Ollama.

Then:

Python/FastAPI → Local Model

instead of:

Python → Cloud LLM API

Live Build

Create a **Multi-Model Gateway**:

Request → Router → Cloud Model OR Local Model → Response

Students choose model based on:

- privacy
- complexity

- availability
- cost

Project 1

Production LLM Application

Must contain:

- Model API
- Local model
- Structured outputs
- Tool calling
- FastAPI
- Conversation handling
- Error handling
- Model routing

PHASE 2 — EMBEDDINGS & VECTOR SEARCH

Days 36–38

DAY 36 — Embeddings from First Principles

This is where RAG actually begins.

Concepts

- Why keyword matching is insufficient
- Semantic meaning
- Embeddings
- Vectors
- Dimensions
- Embedding models
- Query embedding
- Document embedding
- Similarity
- Cosine similarity
- Dot product
- Euclidean distance
- Semantic search

Don't make mathematics overwhelming.

Hands-on

Take sentences:

“Python developer”

“Software engineer experienced in Python”

“Chocolate cake recipe”

Generate embeddings.

Calculate similarities.

Students should **see numerically** why the first two are closer.

Build

Semantic Skill Matcher

JD → embedding
Resume skills → embeddings
Similarity → ranked matches

Critical lesson

An embedding model is **not an LLM response generator**.

DAY 37 — Vector Search & FAISS

Concepts

- Why normal database search differs
- Vector indexes
- Nearest-neighbor search
- KNN
- ANN
- Top-K
- Indexing
- Similarity thresholds
- Vector dimensions
- FAISS
- Index creation
- Adding vectors
- Searching vectors
- Persistence
- Scaling concept

Hands-on

Build semantic search **without LangChain**.

Pipeline:

Documents → Embeddings → FAISS → Query Embedding → Top-K → Results

Live Build

Semantic Document Search Engine

Students can upload a set of documents and search them by meaning.

DAY 38 — Production Vector Databases

Now move from local FAISS toward production infrastructure.

Concepts

- Vector database architecture
- Collection/index
- ID
- Vector
- Metadata
- Namespace/tenant concepts
- Metadata filtering
- Persistence
- Managed vs self-hosted
- Pinecone / Qdrant / Chroma / Weaviate landscape
- Selection trade-offs

Students don't need four identical tutorials.

Choose **one primary production vector DB** and build deeply with it.

Hands-on

Create:

Document → Chunk → Embed → Vector DB

Store:

- vector
- text
- source
- document ID
- page
- category
- metadata

Then query with metadata filters.

Build

Production Semantic Search API

FastAPI → Embedding → Vector DB → Results

PHASE 3 — RAG ENGINEERING

Days 39–45

DAY 39 — RAG from Scratch

No LangChain initially.

Concepts

- Why RAG exists
- Knowledge cutoff
- Private/company knowledge
- Hallucination problem
- Retrieval-Augmented Generation
- Indexing pipeline
- Query pipeline

Architecture

Documents → Parse → Chunk → Embed → Index

Then:

Question → Embed → Retrieve → Context → LLM → Answer

Hands-on

Students manually build every step.

Live Build

PDF Question Answering System V1

Upload PDF → extract text → chunk → embed → store → retrieve → prompt LLM → answer.

Students must understand exactly where the answer came from.

DAY 40 — Document Ingestion & Chunking Engineering

Poor ingestion destroys RAG systems.

Concepts

- PDF
- DOCX
- HTML
- Markdown
- CSV
- Database data
- Parsing
- Cleaning
- Headers/footers
- OCR concept
- Tables
- Metadata
- Chunk size
- Chunk overlap
- Fixed-size chunking
- Recursive chunking
- Sentence chunking
- Semantic chunking
- Parent-child chunking concept
- Document IDs
- Deduplication

Hands-on experiment

Take one document.

Run multiple chunking strategies.

Compare retrieval results.

Build

Reusable Document Ingestion Pipeline

Input → detect → parse → clean → chunk → metadata → embeddings → vector DB.

DAY 41 — Retrieval Engineering

Now students stop thinking:

> “RAG = vector database.”

Concepts

- Dense retrieval
- Sparse retrieval
- BM25
- Keyword search
- Semantic search
- Hybrid search
- Top-K
- Similarity threshold
- Metadata filters
- Retrieval precision/recall intuition
- Query rewriting
- Multi-query retrieval
- Query expansion
- Decomposition

Hands-on

Compare:

Keyword search
Dense search
Hybrid retrieval

on the **same dataset**.

Live Build

Upgrade RAG system with hybrid retrieval.

DAY 42 — Reranking & Advanced Retrieval

Concepts

- Retrieval vs reranking
- Bi-encoder vs cross-encoder intuition
- Rerankers
- Candidate generation
- Reranking top results
- Context selection
- Diversity
- Redundant chunks
- Context compression concept
- Lost-in-the-middle problem

Pipeline

Question
→ retrieve 20
→ rerank
→ choose best 5
→ build context
→ LLM

Hands-on

Add a reranker to the existing RAG project.

Measure difference before/after.

Job interview knowledge

Student should explain why retrieving more chunks doesn't automatically improve RAG.

DAY 43 — Advanced RAG Patterns

Concepts + hands-on

Students implement selected patterns rather than memorize names.

- Conversational RAG
- Query rewriting
- History-aware retrieval
- Multi-query retrieval
- Parent-document retrieval
- Contextual retrieval concepts
- Self-query retrieval
- Router RAG
- Multi-source RAG
- Corrective RAG concept
- Agentic RAG introduction

Live Build

Multi-Source Enterprise RAG

Route query toward appropriate knowledge source.

Example:

HR → HR docs
Technical → engineering docs
Policy → policy docs

DAY 44 — RAG Evaluation, Hallucination & Security

This is **essential for job readiness**.

Most beginner courses stop at:

> “The chatbot answered my PDF.”

We don't.

Concepts

- Why “looks correct” isn't evaluation
- Ground truth
- Evaluation datasets
- Retrieval evaluation
- Precision/recall intuition
- Context relevance
- Answer relevance
- Faithfulness
- Hallucination
- RAG evaluation frameworks/concepts
- LLM-as-judge
- Human evaluation
- Prompt injection
- Indirect prompt injection
- Malicious documents
- Data leakage
- Access control
- Guardrails
- Citations

Hands-on

Create 20–30 test questions.

Run them automatically.

Compare configurations.

Build

RAG Evaluation Pipeline

Configuration A vs B → metrics/results → choose configuration based on evidence.

DAY 45 — Production RAG Build Day

Almost no new theory.

Students combine Days 36–44.

Project 2 — Enterprise Knowledge Assistant

Architecture:

Documents → Ingestion → Cleaning → Chunking → Embeddings → Vector DB → Hybrid Retrieval → Reranking → Context → LLM → Citations → Evaluation

Backend:

FastAPI

Required:

- document upload
- ingestion
- metadata
- vector storage
- retrieval
- reranking
- conversation
- citations
- evaluation
- error handling
- authentication concept
- logging
- deployed API

This should become a **serious portfolio project**, not a notebook.

PHASE 4 — AI AGENTS

Days 46–51

DAY 46 — Agents from Scratch

Start without LangGraph.

Concepts

- LLM application vs workflow vs agent
- Agent loop
- Goal
- Observation
- Reasoning/decision
- Action
- Tool
- Result
- Iteration
- Termination
- ReAct pattern
- Planning
- Tool selection
- Autonomy
- Deterministic vs agentic systems
- When NOT to use agents

Hands-on

Build a manual agent loop.

Tools:

Search
Calculator
Database
External API

Architecture

User → LLM → Decide → Tool → Observe → Decide Again → Final Answer

Students code the loop themselves.

DAY 47 — Production Tool Calling & Agent Reliability

Concepts

- Multiple tools
- Tool descriptions
- Tool schemas
- Tool permissions
- Argument validation
- Tool failures
- Retry
- Timeout
- Fallback
- Maximum iterations
- Infinite loops
- Idempotency concept
- Cost limits
- Human approval
- Dangerous actions
- Structured agent outputs

Hands-on

Create a business agent that can:

Search customer → check data → create/update ticket → send proposed action for approval.

Critical engineering lesson

Never give an autonomous agent unrestricted tools simply because it works in a demo.

DAY 48 — LangGraph & Stateful Agents

Now introduce the framework after students understand the loop.

Concepts

- Graph
- Nodes
- Edges
- State
- Conditional edges
- Cycles
- Agent state
- Tool nodes
- Routing
- Persistence/checkpointing
- State machines
- Deterministic + agentic combinations

Hands-on

Rebuild Day 46 agent using LangGraph.

Build

Research Agent

Plan → Search → Analyze → Validate → Generate → Finish.

Students visualize the graph.

DAY 49 — Memory, Human-in-the-Loop & Long-Running Agents

Concepts

- State vs memory
- Conversation memory
- Short-term memory
- Long-term memory
- Semantic memory concept
- User profile memory
- Persistence
- Checkpoints
- Human approval
- Interrupt/resume
- Long-running tasks
- Recovering after failure

Hands-on

Build an agent that:

remembers user preferences → executes workflow → pauses before important action → user approves → resumes.

Live Build

AI Operations Assistant

DAY 50 — Multi-Agent Systems & Agent Orchestration

Don't make this a CrewAI tutorial.

Concepts

- Why multiple agents?
- When one agent is better
- Supervisor pattern
- Router pattern
- Specialized workers
- Agent handoffs
- Shared state
- Parallel execution
- Sequential execution
- Communication
- Failure propagation
- Cost explosion
- Agent evaluation

Hands-on

Build:

Supervisor Agent

↓

Research Agent
Data Agent
Writer Agent

↓

Supervisor verifies result.

Discuss CrewAI/AutoGen-style approaches, but engineering pattern comes first.

DAY 51 — MCP + Agentic RAG

This should be a heavy practical session.

MCP concepts

- Why MCP exists
- Host
- Client
- Server
- Tools
- Resources
- Prompts
- Transport concept
- Tool discovery
- Permissions/security
- MCP vs ordinary REST API
- MCP in agent architectures

Hands-on

Students:

1. Connect to an existing MCP server.
2. Inspect available tools.
3. Let model use those tools.
4. Build a small custom MCP server.
5. Expose their own tool/resource.
6. Connect agent to it.

Then connect previous RAG system as a tool.

Result

Agent → MCP/Tools → RAG → Database/API → Response

Project 3

Production Stateful AI Agent

Must include:

- multiple tools
- LangGraph
- state
- memory
- RAG

- MCP
- human approval
- error handling
- retries/fallbacks
- logging

PHASE 5 — MODEL CUSTOMIZATION

Days 52–54

DAY 52 — Fine-Tuning Fundamentals + Dataset Engineering

Students should first understand:

Prompting vs RAG vs Fine-Tuning.

Concepts

- Pretraining
- Instruction tuning
- Supervised fine-tuning
- Fine-tuning
- When fine-tuning helps
- When it doesn't
- RAG vs fine-tuning
- Dataset quality
- Instruction-response datasets
- Chat templates
- Training/validation/test split
- Data cleaning
- Deduplication
- Tokenization
- Sequence length
- Overfitting
- Epoch
- Batch
- Learning rate — intuition

Hands-on

Take raw domain data.

Convert it into a proper training dataset.

Validate examples.

Tokenize.

Analyze token lengths.

Output

Fine-Tuning Dataset V1

DAY 53 — LoRA, QLoRA & PEFT: Train a Model

This is a real training day.

Concepts

- Why full fine-tuning is expensive
- PEFT
- LoRA
- Low-rank adapters
- QLoRA
- Quantization during training
- Target modules
- Rank
- Alpha
- Dropout concept
- GPU memory considerations

Hands-on

Students use an appropriate small/open model.

Pipeline:

Base Model
→ Dataset
→ Tokenization
→ LoRA/QLoRA configuration
→ Train
→ Save Adapter

Students should actually watch:

training loss → checkpoints → output.

DAY 54 — Evaluate, Merge, Quantize & Serve Fine-Tuned Model

Training isn't the finish line.

Concepts

- Baseline evaluation
- Before/after comparison
- Validation
- Qualitative evaluation
- Task-specific metrics
- Hallucination/behavior checks
- Adapter loading
- Model merging
- Export
- Quantization
- Serving
- Ollama/vLLM serving concepts
- Model API endpoint
- Versioning

Hands-on

Compare:

Base model vs fine-tuned model

on same evaluation set.

Then:

Fine-tuned model → serve → FastAPI/application → response.

Student outcome

They've now completed:

Dataset → Train → Evaluate → Export → Serve → Integrate

rather than simply knowing the definition of LoRA.

PHASE 6 — MULTIMODAL & VOICE AI

Days 55–56

DAY 55 — Multimodal AI Engineering

Concepts

- Text models

- Vision-language models
- Image understanding
- Documents
- Screenshots
- Charts
- Image inputs
- Multimodal prompts
- Structured extraction
- OCR vs vision models
- Document intelligence
- Multimodal RAG concept
- Image embeddings concept
- Practical limitations

Hands-on

Build:

Multimodal Document Analyst

Input:

PDF/image/invoice/screenshot

↓

Vision-capable model

↓

Structured JSON

↓

Validation

↓

Database

Students should make the model extract structured information, not merely ask “what is in this picture?”

DAY 56 — Voice AI & Realtime Systems

One focused practical session.

Concepts

- Speech-to-text
- Transcription
- Text-to-speech
- Speech-to-speech/realtime models
- Voice activity detection concept
- Streaming audio
- Latency
- Interruptions
- Turn-taking
- Tool calling during voice conversations
- Conversation state
- Telephony architecture concept
- Realtime API/WebSocket concepts
- Voice-agent failure modes

Architecture

Microphone/Call → Audio → STT/Realtime Model → LLM/Agent → Tools → Response → TTS/Audio

Hands-on

Build a simple:

Voice AI Assistant

User speaks → model understands → uses tool if necessary → responds with voice.

Example:

Voice Appointment Assistant

Tools:

check slots → retrieve information → create booking request.

PHASE 7 — EVALUATION, OBSERVABILITY & PRODUCTION AI

Days 57–60

DAY 57 — AI Evaluation Engineering

This should be one of the most important employment-focused classes.

Question

How do you know your AI application is actually good?

Concepts

- Non-deterministic systems
- Test datasets
- Golden datasets
- Offline evaluation
- Online evaluation
- Human evaluation
- LLM-as-judge
- Exact-match/task metrics
- Semantic similarity
- RAG evaluation
- Agent evaluation
- Tool-call correctness
- Task completion
- Hallucination
- Safety evaluation
- Regression testing
- Prompt/model comparison
- A/B testing concept

Hands-on

Take previous application.

Create evaluation suite.

Compare:

Model A vs Model B
Prompt A vs Prompt B
Retriever A vs Retriever B

Generate an evaluation report.

DAY 58 — LLMOps, Observability, Cost & Reliability

Concepts

- AI tracing
- Request traces

- Prompt logging
- Model logging
- Token usage
- Cost
- Latency
- Time-to-first-token
- Errors
- Tool traces
- RAG traces
- Agent trajectories
- Langfuse-style observability
- Feedback
- Production monitoring
- Rate limits
- Retries
- Exponential backoff
- Fallback models
- Caching
- Prompt caching concept
- Semantic caching
- Cost budgets

Hands-on

Instrument one existing AI project.

Students should be able to inspect:

User → Agent → Retrieval → Model → Tool → Model → Response

including:

latency + tokens + cost + failures.

Challenge

Reduce cost/latency without destroying quality.

DAY 59 — Production AI Architecture, Security & Scaling

Now combine their backend and AI knowledge.

Concepts

Architecture

- API gateway concept
- FastAPI
- AI service
- Model providers
- Local model serving
- Vector DB
- PostgreSQL
- Redis
- Background workers
- Queues
- Streaming
- Async processing
- Object storage concept

Reliability

- Timeouts
- Retries
- Circuit-breaker concept
- Fallback models
- Rate limiting
- Idempotency
- Queue-based long jobs

Security

- Authentication
- Authorization
- Tenant isolation
- Secrets
- PII
- Prompt injection
- Data leakage
- Tool permissions
- Output validation
- Model abuse
- Cost abuse

Scaling

- Concurrent users
- Stateless services
- Horizontal scaling concept
- Model throughput
- Batching
- GPU utilization concept
- Caching

- Queue workers

Architecture exercise

Design an AI system for:

10 users → 1,000 users → 10,000 users

Students identify what changes at each stage.

DAY 60 — Ship a Production AI System + Interview/System Design Day

This is the final teaching day.

It should connect **all 60 previous days**, rather than introducing another framework.

Part 1 — Production Deployment

Take one application and ship it.

Architecture:

Client → FastAPI → Authentication → AI Orchestrator → LLM/Local Model → RAG/Vector DB → Agent/Tools → PostgreSQL/Redis → Observability

Students cover:

- Docker
- environment variables
- secrets
- cloud deployment
- database
- vector DB
- logging
- tracing
- health checks
- API testing
- basic CI/CD connection
- production debugging

Part 2 — AI System Design Interview

Give students a requirement:

> "Design an enterprise AI assistant used by 10,000 employees over 1 million internal documents."

They must decide:

- ingestion architecture
- chunking
- embedding model

- vector DB
- retrieval
- reranking
- LLM
- access control
- caching
- evaluation
- monitoring
- latency
- cost
- scaling
- failure handling

The mentor challenges their decisions.

Part 3 — AI Engineering Interview Review

Questions should cover practical decisions:

Why RAG instead of fine-tuning?

Why hybrid retrieval?

When would you use reranking?

Why use LangGraph instead of a simple function chain?

When should you NOT use an agent?

API model vs local model?

How would you reduce hallucinations?

How do you evaluate RAG?

How would you debug an agent that repeatedly calls the wrong tool?

How would you reduce LLM latency?

How do you secure an MCP/tool-enabled agent?

How would you scale a model endpoint?

How do you estimate AI application cost?

This day transitions them directly into the capstone.